



forget everything that happened to be important in the last week and that included your Linux passwords too.

Had you been on Windows and were resolute to make sure that you never have to be denied access to the system just because you changed the password the day before and have completely forgotten it now, most probably you'd have created a password repair disk. But although Windows provides a nice feature for letting you in when you forget your password, often people don't utilize it. It becomes much more difficult to access a Linux system in the same condition because usually, Linux distributions do not offer such functionality. But not to worry, the next time you happen to forget your Linux password, all you need is to follow this guide and you shall be the emperor of your lost Linux Empire 'once again'. But before we take off with the methods, let us imbibe a few things about Linux passwords which will help us understand the steps later.

Where does Linux store its user info?

Linux stores the user details in the `/etc/passwd` file. This file contains each user's details on one individual line. A standard line would look like this:

```
root:x:0:0:root:/
root:/bin/bash
```

In this line, some information is stored. Each piece of information is separated by a colon (:). The first part is the username (root), the second (x) details that the password has been shadowed and is stored in the `/etc/shadow` file. The next two numbers are the user id (0) and group ID (0) of the user. Then we have the full name of the user (root). This field is actually a comment field and can be used for other purposes too. However, most distributions use it to store the

real name of a user. After that we have the home directory of the user, which in this case is (/root). The last field speaks about the location of the executable which this user would use as its command line shell (or simply shell or console).

Yeah, so where is the password?

The interesting part for us in that line is the second field which says that the password is shadowed and stored in `/etc/shadow` file. In fact, the word shadow is used only to say that the actual password is stored in the `/etc/shadow` file. This file (`/etc/shadow`) is actually accessible only to the system administrator which is the 'root' user. So once you open this file, you shall see a similar structure with each line having some sensitive information about a user. Here we show you an example:

```
root:$1$fc214jqk$.fy88wy/
fy88wy/w7J41CinaszNP1:
14913:0:99999:7:::
```

In this line again, we have more than one piece of information separated by colon (:). The first field being the username, the second as the password value and third is the epoch since the password was changed. The next number tells the number of days until which the change is allowed. The following field (99999) defines the number of days after which the user is required to change his password. The next number (7) indicates the number of days before the password expiration when Linux shall display a notice to the user upon login. For example, if the password has to be changed before 28th March 2011 then Linux shall show a notice to the user and ask him to change the password 7 days before March 28, i.e. during 21st to 28th March. The next field is empty but would contain the number of days after which the account gets

disabled if the password were not changed. Exemplarily, if this field contains the value '2' and the last date for changing the password is 28th March 2011 then, the account gets disabled at the end of 30th March 2011. The last field would determine the number of days since the epoch that the account will be disabled regardless of any other activity. If you are wondering what 'epoch' means, it is actually the date Jan-01-1970. The numbers of days mentioned in the shadow file are counted since this date.

What is in the password?

We are concerned about the password. Let us see how Linux makes sense of the password. Let us look at the password value once again:

```
$1$fc214jqk$.fy88wy/
w7J41CinaszNP1
```

If you notice, you shall discover that in total there are three dollar signs. They are actually used to separate (once again) three different but related information contained in the password value. The first field is the text between the first and second dollar sign. In this case, it is '1'. This number is used to indicate which algorithm was used to create the hash. Here, it would be the well known MD5 algorithm because the number 1 at this position corresponds to MD5.

The second value is called 'salt' (reminds you of Angelina and some action, eh?) and is located between the second and third dollar signs. In this case it is 'fc214jqk'. Just like in real world, salt is used to change the taste of food; in this case, it is used to alter the output of the algorithm being used to create the password hash. The salt is random and is automatically generated by Linux whenever you create or change a password. The reason behind use of salt is

quite interesting and we will talk about it shortly.

The third value in the password value field is the password hash which has been generated by the algorithm in accordance with the salt value and the password that the user inputs. This is the string between the third dollar sign and the next colon symbol on the line. It is this value which finally matters. The algorithm and the salt are used to engender the password hash.

Windows passwords vs. Linux passwords

If you look carefully, you shall see that the way Linux stores its password is both simple and intelligent. The files are locked with appropriate permission set and there is no mysticism involved. Simple hashes and salts; that's all! But in some ways it is more intelligent a method than what Microsoft uses in its Windows Operating System. There are two good features in the password storage technique used by Linux which makes it better than that of Windows.

The first of them is the possibility of using any algorithm you want to use. Linux has quite a few built-in algorithms for hashing the passwords and any of them can be used. So there is no way an attacker would be able to be sure of what hashing technique one might be using on the machine. The flexibility is that you can use one hashing technique (say MD5) for one user account and another technique (say Blowfish) for another.

The second is the use of a salt. Since salt is a random string, there is no way to predict it, unless you have already read it in the `/etc/shadow` file. The salt is used to change the way the hashing algorithm works to create the password hash. Same algorithm and same user input with different salt values will give birth to different hashes.